cka

October 6, 2026

1 Centered Kernel Alignment — CKA (Kornblith, Norouzi, Lee, Hinton, 2019)

Métrica secundaria de similitud de representaciones (bloque A.1 del capítulo de ViT), pensada para no razonar en círculo con la penalización (que ya usa producto punto entre cabezas) — CKA es una forma distinta, más robusta, de medir qué tan parecidas son dos representaciones.

Sean $X \in \mathbb{R}^{n \times p_1}$ e $Y \in \mathbb{R}^{n \times p_2}$ las activaciones de n ejemplos en dos representaciones (p. ej. dos cabezas, o la misma cabeza en dos modelos). Con kernels $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$, $L_{ij} = l(\mathbf{y}_i, \mathbf{y}_j)$ y matriz de centrado $H = I_n - \frac{1}{n}\mathbf{1}\mathbf{1}^T$:

$$\mathrm{HSIC}(K, L) = \frac{1}{(n-1)^2}\,\mathrm{tr}(KHLH), \qquad \mathrm{CKA}(K, L) = \frac{\mathrm{HSIC}(K, L)}{\sqrt{\mathrm{HSIC}(K, K)\,\mathrm{HSIC}(L, L)}}$$

Con kernel lineal ($K = XX^T$, $L = YY^T$, columnas centradas) esto se reduce a una forma cerrada barata de calcular:

$$\mathrm{CKA}(XX^T, YY^T) = \frac{\|Y^T X\|_F^2}{\|X^T X\|_F \,\|Y^T Y\|_F}$$

Propiedad clave (Sección 2 del paper): CKA es invariante a transformación **ortogonal** e isotrópica, pero **no** a transformación lineal invertible arbitraria — a diferencia de CCA, que sí es invariante a cualquier transformación invertible y por eso colapsa (da 1) cuando hay más neuronas que ejemplos ($p \geq n$).

[1]:
```
import torch


def linear_cka(X: torch.Tensor, Y: torch.Tensor) -> torch.Tensor:
    """
    Centered Kernel Alignment (Kornblith et al., 2019) con kernel lineal.

    X: (..., n, p1), Y: (..., n, p2) -- n ejemplos, p1/p2 features por␣
 ↪representación.
    Se centran las columnas (media 0 por feature) antes de calcular la␣
 ↪similitud.

    CKA(XX^T, YY^T) = ||Y^T X||_F^2 / (||X^T X||_F * ||Y^T Y||_F)
```

```
    Devuelve un escalar en [0, 1]: 1 si las representaciones son idénticas
    salvo rotación/escala isotrópica, cerca de 0 si son independientes.
    """
    X = X - X.mean(dim=-2, keepdim=True)
    Y = Y - Y.mean(dim=-2, keepdim=True)

    cross_term = torch.linalg.matrix_norm(Y.transpose(-2, -1) @ X) ** 2
    xtx_norm = torch.linalg.matrix_norm(X.transpose(-2, -1) @ X)
    yty_norm = torch.linalg.matrix_norm(Y.transpose(-2, -1) @ Y)

    return cross_term / (xtx_norm * yty_norm)
```

1.1 Pruebas con matrices de referencia

Cuatro casos que validan las propiedades del paper, no solo que la función corre:

1. **Auto-similitud**: $\mathrm{CKA}(X, X) = 1$ exacto (identidad trivial).
2. **Invariancia a rotación**: $\mathrm{CKA}(X, XQ) \approx 1$ para Q ortogonal — CKA no distingue representaciones que difieren solo en una rotación de sus ejes.
3. **Sensibilidad a transformación no ortogonal**: $\mathrm{CKA}(X, XA)$ para A diagonal con escalas muy distintas por eje (invertible, pero no ortogonal ni isotrópica) da un valor claramente menor a 1 — a diferencia de CCA, que sería invariante también aquí.
4. **Representaciones independientes**: $\mathrm{CKA}(X, Z)$ con Z aleatorio e independiente de X da un valor bajo, cercano a 0.

[2]:
```
import math

torch.manual_seed(0)
n, p = 200, 20
X = torch.randn(n, p)

cka_self = linear_cka(X, X).item()
print(f"[auto-similitud]         CKA(X, X)   = {cka_self:.6f}  (esperado 1.0)")
assert math.isclose(cka_self, 1.0, abs_tol=1e-5)
```

```
[auto-similitud]         CKA(X, X)   = 1.000000  (esperado 1.0)
```

[3]:
```
Q, _ = torch.linalg.qr(torch.randn(p, p))  # Q ortogonal: Q^T Q = I
Y_rot = X @ Q

cka_rot = linear_cka(X, Y_rot).item()
print(f"[rotación ortogonal]     CKA(X, XQ)  = {cka_rot:.6f}  (esperado ~1.0)")
assert math.isclose(cka_rot, 1.0, abs_tol=1e-4)
```

```
[rotación ortogonal]     CKA(X, XQ)  = 1.000000  (esperado ~1.0)
```

[4]:
```
A = torch.diag(torch.linspace(0.01, 10, p))  # invertible, pero no ortogonal ni␣
↪isotrópica
```

```
Y_scaled = X @ A

cka_scaled = linear_cka(X, Y_scaled).item()
print(f"[escala no isotrópica]   CKA(X, XA)  = {cka_scaled:.6f}  (< 1, a␣
 ↪diferencia de CCA)")
assert cka_scaled < 0.95
```

```
[escala no isotrópica]   CKA(X, XA)  = 0.750387  (< 1, a diferencia de CCA)
```

[5]:
```
Z = torch.randn(n, p)  # independiente de X

cka_indep = linear_cka(X, Z).item()
print(f"[independientes]         CKA(X, Z)   = {cka_indep:.6f}  (cercano a 0)")
assert cka_indep < 0.3

print("\nOK: linear_cka coincide con las propiedades esperadas (Kornblith et al.
 ↪, 2019).")
```

```
[independientes]         CKA(X, Z)   = 0.081679  (cercano a 0)

OK: linear_cka coincide con las propiedades esperadas (Kornblith et al., 2019).
```